

MPonline

Advanced Software Engineering & Development Internship

Batch 11(A) – Sulakshana Deore

Project Report

Library Management System

Submitted By

Name: PRATYAKSHA SINGH

Registration Number: 23BAI10345

Enrollment Number: IN26013448

TABLE OF CONTENTS

1. Introduction	4
2. Project Objectives	5
3. System Requirements	7
4. Technology Stack	9
5. System Architecture & Design	11
6. Database Design & Data Models	13
7. Module Design	16
7.1 Authentication & Session Management	16
7.2 Dashboard Module	18
7.3 Books Module	20
7.4 Newspapers Module	23
7.5 Magazines Module	25
7.6 Students Module	27
7.7 Librarians Module	29
7.8 Borrow & Return Module	31
7.9 Profile Module	33
7.10 About Us & Contact Us	34
8. User Interface Design	36
8.1 Design System & Theming	36
8.2 Login Interface Screenshots	38
8.3 Dashboard Screenshots	39
8.4 Module Screenshots	41
9. CSS Styling & Design Tokens	43
10. Security Implementation	45
11. Testing & Validation	46
12. Future Enhancements	48
13. Conclusion	49
14. References	50

1. Introduction

The Library Management System (LMSystem) is a comprehensive, enterprise-grade web application developed to digitize and streamline the complete workflow of modern academic library operations. Built as part of the MPonline Advanced Software Engineering & Development Internship (Batch 11A), this project demonstrates proficiency in full-stack web development using industry-standard technologies.

In today's digital era, efficient management of library resources—including books, newspapers, magazines, student memberships, and staff assignments—is critical for any academic institution. Traditional paper-based systems are prone to errors, data loss, and inefficiency. This project addresses these challenges by providing a robust, intuitive, and visually stunning digital solution.

The application serves as a centralized platform where administrators, librarians, and students can interact with the library's catalog, manage borrowing and returning of books, track memberships, and access institutional information—all through a beautifully designed web interface that supports both light and dark themes.

Key differentiators of this system include:

- Role-based access control with secure session management
- Interactive data visualization with Chart.js analytics on the dashboard
- Complete CRUD operations across all entity modules
- Indian Rupee (₹ / INR) currency support throughout the system
- Premium UI/UX with dual-theme (light/dark) support and custom design tokens
- Responsive layout optimized for modern screen resolutions
- Real-time overdue fine calculator and CSV/JSON data export functionality

This report provides a comprehensive walkthrough of the project, covering objectives, system architecture, module design with code snippets, user interface screenshots, testing methodology, and future enhancement roadmaps.

2. Project Objectives

The primary objectives of this Library Management System project are outlined below:

2.1 Primary Objectives

- To develop a fully functional web-based Library Management System using Node.js and Express.js framework.
- To implement secure user authentication and session-based authorization with role-based access control (RBAC).
- To provide comprehensive CRUD (Create, Read, Update, Delete) operations for all library entities including Books, Newspapers, Magazines, Students, and Librarians.
- To design and implement an interactive, data-driven administrative dashboard with real-time statistics and visual analytics.
- To create a premium, professional user interface with a custom CSS design system supporting dual themes (light and dark mode).
- To integrate Indian Rupee (₹/INR) as the standard currency across all financial fields.

2.2 Secondary Objectives

- To implement search and filter functionality across all catalog modules.
- To develop a book borrowing and return workflow with overdue fine estimation capabilities.
- To provide data export functionality in CSV and JSON formats for administrative reporting.
- To create informational pages (About Us, Contact Us) for institutional presence.
- To implement user profile management with editable account details.
- To ensure zero horizontal scroll and responsive design across all screen sizes.

2.3 Learning Objectives

- Understanding the MVC (Model-View-Controller) architectural pattern in web development.

- Hands-on experience with server-side rendering using EJS (Embedded JavaScript) templating.
- Proficiency in Express.js middleware, routing, and session management.
- Mastery of CSS custom properties (CSS variables) for scalable theming systems.
- Practical application of Bootstrap 5 framework with custom overrides.

3. System Requirements

3.1 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB or higher
Storage	500 MB free disk space	1 GB SSD recommended
Display	1366 × 768 resolution	1920 × 1080 (Full HD) or higher
Network	Internet for CDN assets	Stable broadband connection

3.2 Software Requirements

Software	Version	Purpose
Node.js	v18.x or higher	JavaScript runtime environment
npm	v9.x or higher	Package manager for dependencies
Web Browser	Chrome 100+ / Edge 100+	Client-side rendering and interaction
Code Editor	VS Code (recommended)	Development and debugging
Git	v2.40+	Version control and collaboration
Operating System	Windows 10/11, macOS, Linux	Development platform

3.3 npm Dependencies

Package	Version	Purpose
express	^5.2.1	Web application framework
ejs	^6.0.1	Embedded JavaScript templating engine
express-session	^1.19.0	Server-side session management
Bootstrap 5	5.3.0 (CDN)	Responsive CSS framework
Bootstrap Icons	1.10.5 (CDN)	Icon library
Chart.js	4.x (CDN)	Interactive data visualization

4. Technology Stack

The Library Management System is built using a carefully selected technology stack that balances simplicity, performance, and scalability. Each technology was chosen for its specific strengths in the context of this application.

4.1 Backend Technologies

Node.js: A cross-platform, open-source JavaScript runtime environment built on Chrome's V8 engine. It enables server-side JavaScript execution with an event-driven, non-blocking I/O model that makes it highly efficient for web applications.

Express.js: A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies routing, middleware integration, and HTTP request handling.

express-session: A middleware module for Express that provides session management functionality. It stores session data server-side and uses cookies for session identification, enabling features like login persistence and flash messages.

crypto: A built-in Node.js module used for SHA-256 password hashing during user registration, ensuring passwords are never stored in plain text.

4.2 Frontend Technologies

EJS (Embedded JavaScript): A simple templating language that generates HTML markup with plain JavaScript. It enables server-side rendering with dynamic data injection, conditional rendering, and template composition through includes and partials.

Bootstrap 5: A powerful, feature-packed frontend toolkit used for responsive grid layout, utility classes, and pre-built components. Custom CSS overrides ensure a unique design identity.

Custom CSS3 Design System: A comprehensive design token system using CSS custom properties (variables) that enables seamless theme switching between light and dark modes. The system defines colors, shadows, typography, and component styles.

Chart.js: An open-source JavaScript library for data visualization, used to render interactive line charts on the administrative dashboard showing monthly borrowing and return trends.

4.3 Typography

- Cinzel — Used for headings and branding elements (elegant serif font)
- Plus Jakarta Sans — Primary body text (modern sans-serif)
- Montserrat — Table headers and labels
- JetBrains Mono — Code and monospace elements
- Outfit — Secondary UI elements

5. System Architecture & Design

The Library Management System follows a Model-View-Controller (MVC) architectural pattern adapted for the Node.js Express ecosystem. This separation of concerns ensures maintainability, testability, and scalability of the codebase.

5.1 Architecture Overview

The system architecture consists of three primary layers:

- **Presentation Layer (Views):** EJS templates rendered server-side with Bootstrap 5 and custom CSS.
- **Application Layer (Controllers/Routes):** Express.js route handlers managing business logic, validation, and data flow.
- **Data Layer (Models):** In-memory JavaScript arrays simulating database entities for rapid prototyping.

5.2 Directory Structure

File: *Project Directory Layout*

```
Library Management System/
├── views_ejs/                # EJS View Templates
│   ├── header.ejs           # Shared navigation bar
│   ├── footer.ejs           # Shared footer component
│   ├── login.ejs            # Authentication page
│   ├── register.ejs         # User registration page
│   ├── dashboard.ejs        # Admin dashboard
│   ├── profile.ejs          # User profile page
│   ├── books_index.ejs      # Books catalog listing
│   ├── books_create.ejs     # Add new book form
│   ├── books_edit.ejs       # Edit book form
│   ├── books_delete.ejs     # Delete book confirmation
│   ├── books_details.ejs    # Book detail view
│   ├── newspaper_index.ejs  # Newspapers listing
│   ├── newspaper_create.ejs # Add newspaper form
│   ├── newspaper_edit.ejs   # Edit newspaper form
│   ├── magazine_index.ejs   # Magazines listing
│   ├── magazine_create.ejs  # Add magazine form
│   ├── magazine_edit.ejs    # Edit magazine form
│   ├── student_index.ejs    # Students directory
│   ├── student_create.ejs   # Add student form
│   ├── student_edit.ejs     # Edit student form
│   ├── librarian_index.ejs  # Librarians directory
│   ├── librarian_create.ejs # Add librarian form
│   ├── librarian_edit.ejs   # Edit librarian form
│   └── borrow_create.ejs    # Borrow book form
```

```

|   |— borrow_return.ejs      # Return book form
|   |— about.ejs             # About Us page
|   |— contact.ejs           # Contact Us page
|— public/
|   |— css/style.css          # Complete Design System
|   |— images/                # Logo and graphic assets
|— screenshots/               # Application demo screenshots
|— server.js                  # Main Express Server (1067 lines)
|— package.json               # npm Dependencies Manifest
|— README.md                  # Project Documentation

```

5.3 Request-Response Flow

1. The client browser sends an HTTP request to the Express server.
2. Express middleware processes the request (session validation, flash messages, authentication guard).
3. The route handler executes business logic (CRUD operations on in-memory data).
4. The EJS view engine renders the template with injected data.
5. The rendered HTML response is sent back to the client.
6. Client-side JavaScript handles theme toggling, chart rendering, and interactive filters.

6. Database Design & Data Models

The application uses in-memory JavaScript arrays as data stores, simulating database tables. This approach enables rapid prototyping without requiring a database server. Each entity is modeled as a JavaScript object with well-defined properties.

6.1 Users / Login Table

Field	Type	Description
id	Integer	Primary key, auto-incremented
username	String	Unique login identifier
password	String	SHA-256 hashed password
role	String	System Administrator / Student User / Librarian User
name	String	Display name of the user

File: *server.js* – User Data

```
let logintab = [
  { id: 1, username: 'admin', password: '12345',
    role: 'System Administrator', name: 'Pratyaksha Singh' },
  { id: 2, username: 'mycodingproject', password: 'myc546',
    role: 'Student User', name: 'Pratyaksha Member' },
  { id: 3, username: 'my', password: 'myc',
    role: 'Librarian User', name: 'Rajesh Kumar' },
];
```

6.2 Books Entity

Field	Type	Description
bookId	Integer	Primary key
title	String	Book title
author	String	Author name
genre	String	Book genre/category
isbn	String	ISBN identifier
publishedDate	String	Publication date
price	Number	Price in ₹ INR
isAvailable	Boolean	Availability status
coverImage	String	URL to cover thumbnail
description	String	Brief description

6.3 Newspapers Entity

Field	Type	Description
newspaperId	Integer	Primary key
title	String	Newspaper title
publisher	String	Publishing organization

language	String	Publication language
category	String	Category (National Daily, Business)
dailyPrice	Number	Daily price in ₹ INR
subscriptionType	String	Daily / Weekly / Monthly
publishDate	String	Publication date
isAvailable	Boolean	Availability status

6.4 Magazines Entity

Field	Type	Description
magazineId	Integer	Primary key
title	String	Magazine title
publisher	String	Publishing organization
issueNumber	String	Issue identifier (e.g., August 2026)
genre	String	Genre domain
price	Number	Subscription price in ₹ INR
frequency	String	Weekly / Fortnightly / Monthly
publishDate	String	Publication date
isAvailable	Boolean	Availability status

6.5 Students Entity

Field	Type	Description
studentId	Integer	Primary key
studentName	String	Full name
email	String	Email address
phone	String	Contact number
membershipDate	String	Date of membership
borrowCount	Number	Total books borrowed

6.6 Librarians Entity

Field	Type	Description
librarianId	Integer	Primary key
name	String	Full name
age	Integer	Age in years
phone	String	Contact number
employeeId	String	Staff ID (e.g., LIB-001)
department	String	Assigned department
experience	String	Years of experience
shift	String	Working shift

6.7 Borrow Records Entity

Field	Type	Description
borrowRecordId	Integer	Primary key
bookId	Integer	Foreign key to Books

borrowerName	String	Name of borrower
borrowerEmail	String	Email of borrower
phone	String	Contact number
borrowDate	String	Date and time of borrowing
returnDate	String/null	Date of return (null if active)

7. Module Design

This chapter presents the detailed design and implementation of each functional module in the Library Management System, with relevant code snippets demonstrating the Express.js route handlers and EJS template logic.

7.1 Authentication & Session Management

The authentication module provides secure login/logout functionality with session-based persistence. It supports multiple user roles with appropriate access control.

Key Features:

- SHA-256 password hashing for secure credential storage
- Session-based authentication with 1-hour expiry
- Authentication guard middleware protecting all routes
- Flash message system for success/error notifications
- User registration with duplicate detection

File: *server.js* – Authentication Guard Middleware

```
// AUTHENTICATION GUARD MIDDLEWARE
function requireAuth(req, res, next) {
  if (req.session.isLoggedIn) {
    return next();
  }
  res.redirect('/Login');
}

// Apply to all routes below this line
app.use(requireAuth);
```

File: *server.js* – Login Verification

```
app.post('/Login/Verify', (req, res) => {
  const { username, password } = req.body;
  const hashedPassword = crypto.createHash('sha256')
    .update(password).digest('hex');
  const user = logintab.find(u =>
    u.username === username &&
    (u.password === password ||
     u.password === hashedPassword)
  );
  if (user) {
    req.session.isLoggedIn = true;
```

```
    req.session.user = user;
    flashMessage = `Welcome back, ${user.name}!`;
    res.redirect('/Dashboard');
  } else {
    res.render('login', {
      message: 'Invalid username or password.'
    });
  }
});
```

7.2 Dashboard Module

The dashboard serves as the central hub of the application, providing administrators with a comprehensive overview of library operations through visual analytics, quick-access navigation, and recent activity feeds.

Dashboard Components:

- Welcome banner with personalized greeting and किताब branding logo
- Quick action buttons for primary navigation (Books, Members, Newspapers, Magazines)
- Statistical metrics row: Total Books, Active Borrowings, Total Members, Publications
- Interactive Chart.js line graph showing monthly borrowing and return trends
- Recent activity timeline with categorized events
- Popular Reads section displaying top book covers

File: *server.js – Dashboard Route*

```
app.get(['/Dashboard', '/Dashboard/Index'], (req, res) => {
  const topBooks = books.slice(0, 6);
  const model = {
    totalBooks: books.length,
    totalStudents: students.length,
    totalBorrowings: borrowRecords.filter(
      b => !b.returnDate).length,
    totalPublications: newspapers.length + magazines.length,
    monthlyTrends: {
      labels: ['Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul'],
      borrowed: [32, 45, 38, 52, 60, 44],
      returned: [28, 40, 36, 48, 55, 39]
    },
    topBooks: topBooks
  };
  res.render('dashboard', { model, user: req.session.user });
});
```

7.3 Books Module

The Books module is the most feature-rich module in the application. It provides a visual card-based catalog with cover thumbnails fetched from the Open Library API, search functionality, category filters, and complete CRUD operations.

Features:

- Visual book cards with high-resolution cover images from Open Library
- Availability badges (Available / Out of Stock) with color-coded indicators
- Search by title, author, or ISBN
- Filter tabs: All Books, Available, Borrowed, Under ₹500, Over ₹500
- Overdue Fine Calculator (₹10/day rate)
- CSV and JSON export functionality
- Inline View, Edit, Delete, and Borrow action icons

File: *server.js* – Books Index Route

```
app.get(['/Books', '/Books/Index'], (req, res) => {
  let searchQuery = req.query.searchQuery || '';
  // Sort books: available first
  books.sort((a, b) => {
    if (a.isAvailable === b.isAvailable) return 0;
    return a.isAvailable ? -1 : 1;
  });
  let filteredBooks = books.map(b => {
    const activeBorrow = borrowRecords.find(
      br => br.bookId === b.bookId && !br.returnDate);
    return { ...b, activeBorrowRecordId:
      activeBorrow ? activeBorrow.borrowRecordId : null };
  });
  if (searchQuery.trim()) {
    const query = searchQuery.trim().toLowerCase();
    filteredBooks = filteredBooks.filter(b =>
      b.title.toLowerCase().includes(query) ||
      b.author.toLowerCase().includes(query));
  }
  res.render('books_index', {
    books: filteredBooks, searchQuery });
});
```

File: *server.js* – Books Create Route

```
app.post('/Books/Create', (req, res) => {
  const { title, author, isbn, publishedDate, price } = req.body;
  const newId = books.length > 0 ?
    Math.max(...books.map(b => b.bookId)) + 1 : 1;
```

```
books.push({
  bookId: newId, title, author, isbn, publishedDate,
  price: parseFloat(price) || 500,
  isAvailable: true,
  coverImage: 'https://via.placeholder.com/300x400'
});
flashMessage = `Successfully registered book: ${title}.`;
res.redirect('/Books/Index');
});
```

7.4 Newspapers Module

The Newspapers module manages daily periodical publications with support for multiple languages (English, Hindi), categories, and subscription types. Data is presented in an elegant table format with the custom design system.

Table Columns:

- Title, Publisher, Language, Category, Daily Price (₹), Subscription Type, Publish Date, Actions

File: *server.js* – Newspaper Create Route

```
app.post('/Newspaper/Create', (req, res) => {
  const { title, publisher, language, category,
    dailyPrice, subscriptionType, publishDate } = req.body;
  const newId = newspapers.length > 0 ?
    Math.max(...newspapers.map(n => n.newspaperId)) + 1 : 1;
  newspapers.push({
    newspaperId: newId, title, publisher,
    language: language || 'English',
    category: category || 'General',
    dailyPrice: parseFloat(dailyPrice) || 6,
    subscriptionType: subscriptionType || 'Daily Edition',
    publishDate: publishDate || new Date().toISOString()
      .substring(0, 10),
    isAvailable: true
  });
  flashMessage = `Registered newspaper: ${title}.`;
  res.redirect('/Newspaper/Index');
});
```

7.5 Magazines Module

The Magazines module handles periodical publications with varying frequencies (Weekly, Fortnightly, Monthly). It includes genre-based filtering and search capabilities.

Filter Tabs:

- All Genres, Current Affairs, Business & Wealth, Science & Nature, Politics & Culture, Weekly, Monthly

File: *server.js* – Magazine Index Route

```
app.get(['/Magazine', '/Magazine/Index'], (req, res) => {
  let searchQuery = req.query.searchQuery || '';
  let filteredMagazines = [...magazines];
  if (searchQuery.trim()) {
    const query = searchQuery.trim().toLowerCase();
    filteredMagazines = filteredMagazines.filter(m =>
      m.title.toLowerCase().includes(query) ||
      m.publisher.toLowerCase().includes(query) ||
      m.genre.toLowerCase().includes(query));
  }
  res.render('magazine_index', {
    magazines: filteredMagazines, searchQuery });
});
```

7.6 Students Module

The Students module provides a complete directory of registered library members. It supports search by name, email, or phone number, with CRUD operations for managing student records.

File: *server.js* – Student CRUD Operations

```
// CREATE
app.post('/Student/Create', (req, res) => {
  const { studentName, email, phone } = req.body;
  const newId = students.length > 0 ?
    Math.max(...students.map(s => s.studentId)) + 1 : 1;
  students.push({ studentId: newId, studentName, email, phone });
  flashMessage = `Added new student: ${studentName}.`;
  res.redirect('/Student/Index');
});

// UPDATE
app.post('/Student/Edit/:id', (req, res) => {
  const id = parseInt(req.body.studentId || req.params.id);
  const student = students.find(s => s.studentId === id);
  if (student) {
    student.studentName = req.body.studentName;
    student.email = req.body.email;
    student.phone = req.body.phone;
  }
  res.redirect('/Student/Index');
});

// DELETE
app.get('/Student/Delete/:id', (req, res) => {
  const id = parseInt(req.params.id);
  students = students.filter(s => s.studentId !== id);
  res.redirect('/Student/Index');
});
```

7.7 Librarians Module

The Librarians module manages the operational staff directory, including personal details, department assignments, experience levels, and shift schedules. The interface mirrors the Students module for UI consistency.

File: *server.js* – Librarian Create Route

```
app.post('/Librarian/Create', (req, res) => {
  const { name, age, phone } = req.body;
  const newId = librarians.length > 0 ?
    Math.max(...librarians.map(l => l.librarianId)) + 1 : 1;
  librarians.push({
    librarianId: newId, name,
    age: parseInt(age), phone
  });
  flashMessage = `Added librarian: ${name}.`;
  res.redirect('/Librarian/Index');
});
```

7.8 Borrow & Return Module

The Borrow and Return module implements the core library workflow: issuing books to members and processing returns. It includes validation to prevent double-borrowing and automatically manages book availability status.

Workflow:

1. User clicks "Borrow" on an available book card.
2. System validates availability and presents the borrow form.
3. User fills borrower details (name, email, phone) and submits.
4. System creates a borrow record, marks book as unavailable.
5. For returns, system marks the return date and restores availability.

File: *server.js* – Borrow Create

```
app.post('/Borrow/Create', (req, res) => {
  const bookId = parseInt(req.body.bookId);
  const book = books.find(b => b.bookId === bookId);
  if (!book || !book.isAvailable) {
    return res.render('shared_notfound', {
      errorMessage: 'Book not available.' });
  }
  const newBorrowId = borrowRecords.length > 0 ?
    Math.max(...borrowRecords.map(
      br => br.borrowRecordId)) + 1 : 101;
  borrowRecords.push({
    borrowRecordId: newBorrowId,
    bookId: book.bookId,
    borrowerName: req.body.borrowerName,
    borrowerEmail: req.body.borrowerEmail,
    phone: req.body.phone,
    borrowDate: new Date().toISOString()
      .replace('T', ' ').substring(0, 16),
    returnDate: null
  });
  book.isAvailable = false;
  flashMessage = `Borrowed: ${book.title}.`;
  res.redirect('/Books/Index');
});
```

7.9 Profile Module

The Profile module allows authenticated users to view and edit their account details, including full name and email address. The username field is read-only to maintain session integrity.

Profile Page Features:

- User avatar with initials
- Role badge (System Administrator / Student / Librarian)
- Editable full name and email fields
- Favorite genres and books section
- Member since date and books read counter

File: *server.js* – Profile Update

```
app.post('/Profile/Update', (req, res) => {
  const { name, email } = req.body;
  const userIndex = logintab.findIndex(
    u => u.username === req.session.user.username);
  if (userIndex !== -1) {
    logintab[userIndex].name = name;
    logintab[userIndex].email = email;
    req.session.user = logintab[userIndex];
  }
  flashMessage = 'Profile updated successfully!';
  res.redirect('/Profile');
});
```

7.10 About Us & Contact Us

The About Us page presents the organizational history, core mission statements, and administrative vision of Kitaab Library. The Contact Us page provides an interactive inquiry form and displays the library's physical address, phone helpline, and operational hours in IST.

File: *server.js* – Contact Submission

```
app.post('/Contact/Submit', (req, res) => {  
  const { name, email, subject, message } = req.body;  
  flashMessage = `Thank you, ${name || 'User'}!  
    Your inquiry has been dispatched to  
    ipratyaksha.works@gmail.com.`;  
  res.redirect('/Contact');  
});
```

8. User Interface Design

The user interface of the Library Management System has been meticulously crafted with a premium, enterprise-grade aesthetic. The design system supports seamless switching between light and dark themes, with every component respecting the active color palette.

8.1 Design System & Theming

The application uses a comprehensive CSS custom properties (CSS variables) based design system. All colors, shadows, typography, and component styles are defined as tokens in the `:root` scope and overridden in the `[data-theme="dark"]` scope.

Light Theme Palette:

- Background: Soft off-white (#F4F7F6) with slate blue card containers (#cbd5e1)
- Text: Dark charcoal (#1a1f1d) with sage secondary (#4a5d54)
- Accent: Forest green buttons (#1B5E20) with gold highlights

Dark Theme Palette:

- Background: Deep navy blue (#060B19) with translucent forest green cards
- Text: Pure white (#ffffff) with muted sage (#9DB2AA)
- Accent: Emerald green (#2E6F57) with cyan glow effects

8.2 Login Interface Screenshots

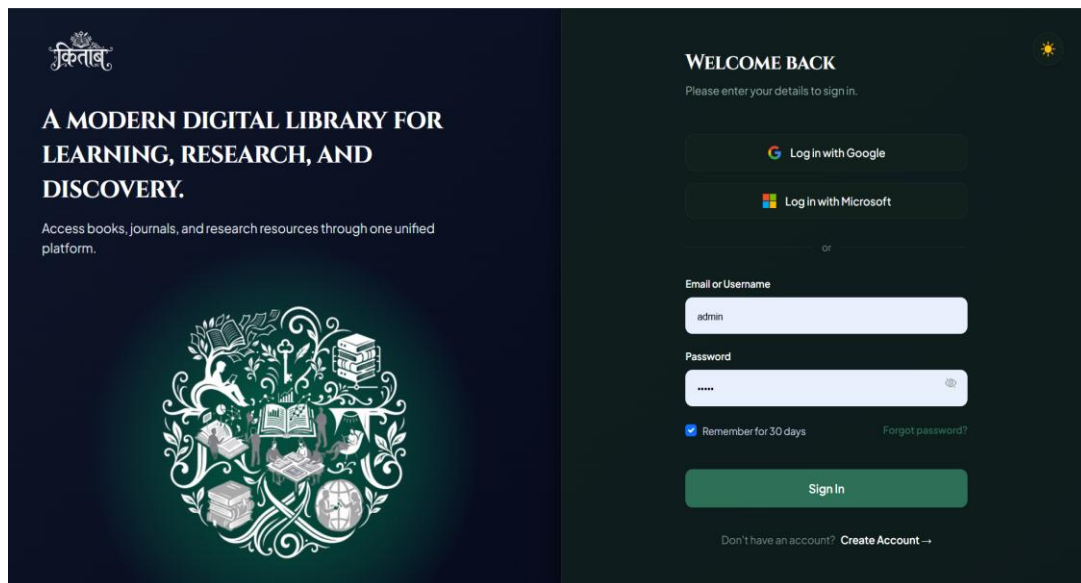


Figure: Login Page — Dark Theme

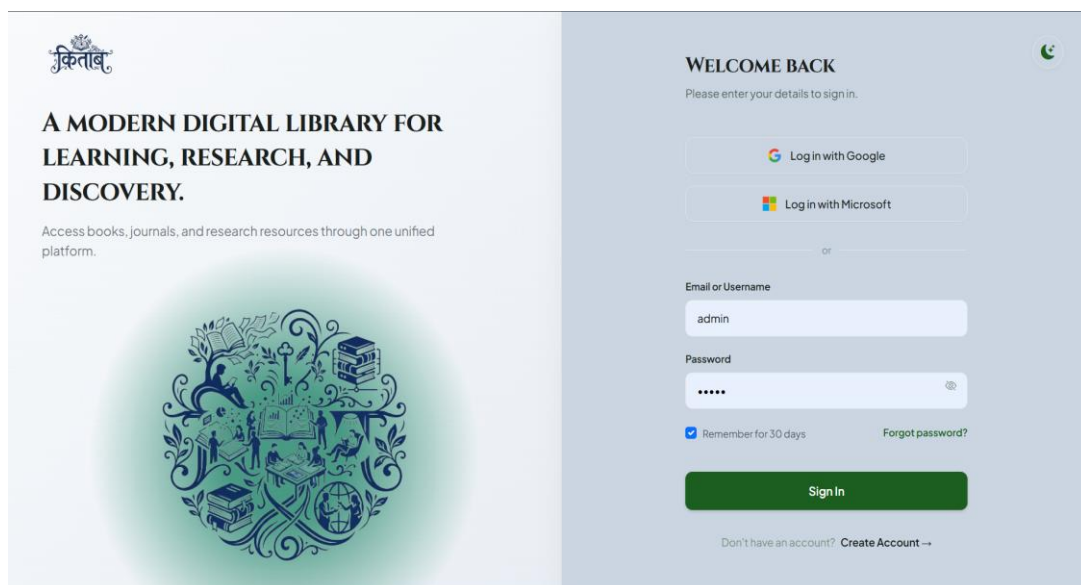


Figure: Login Page — Light Theme

8.3 Dashboard Screenshots

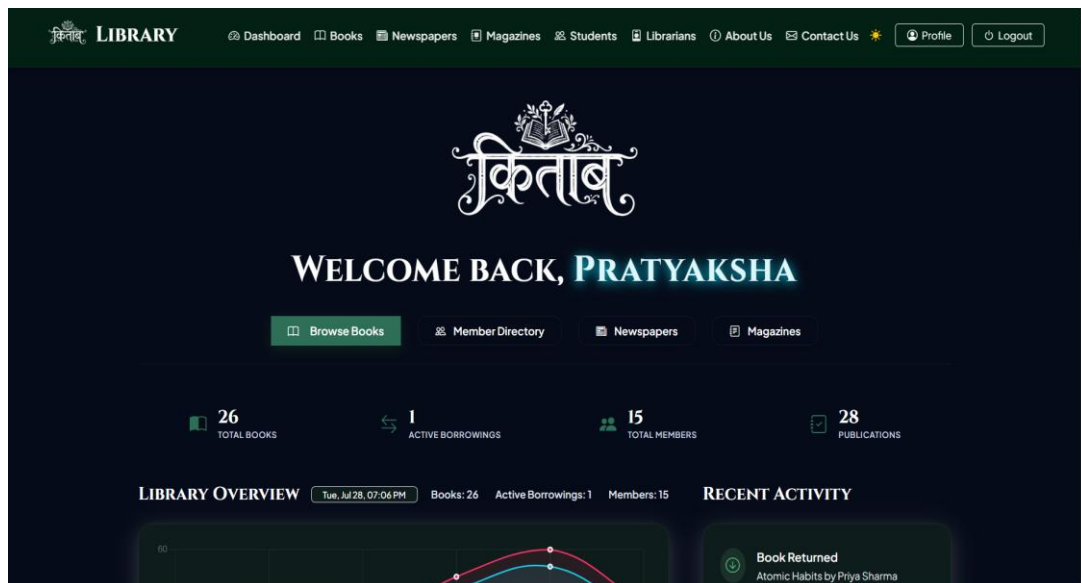


Figure: Dashboard Top Section — Dark Theme

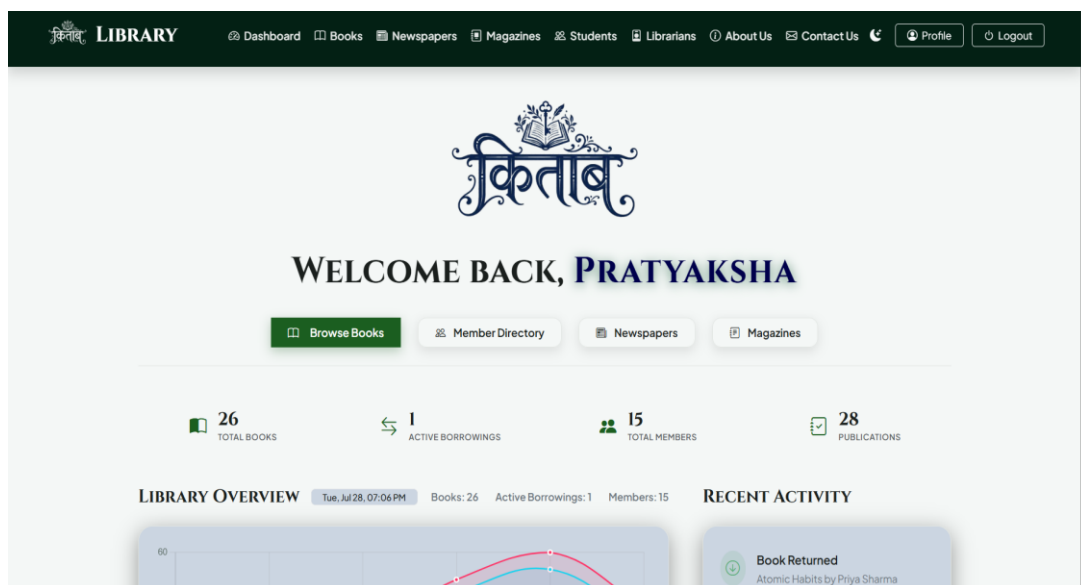


Figure: Dashboard Top Section — Light Theme

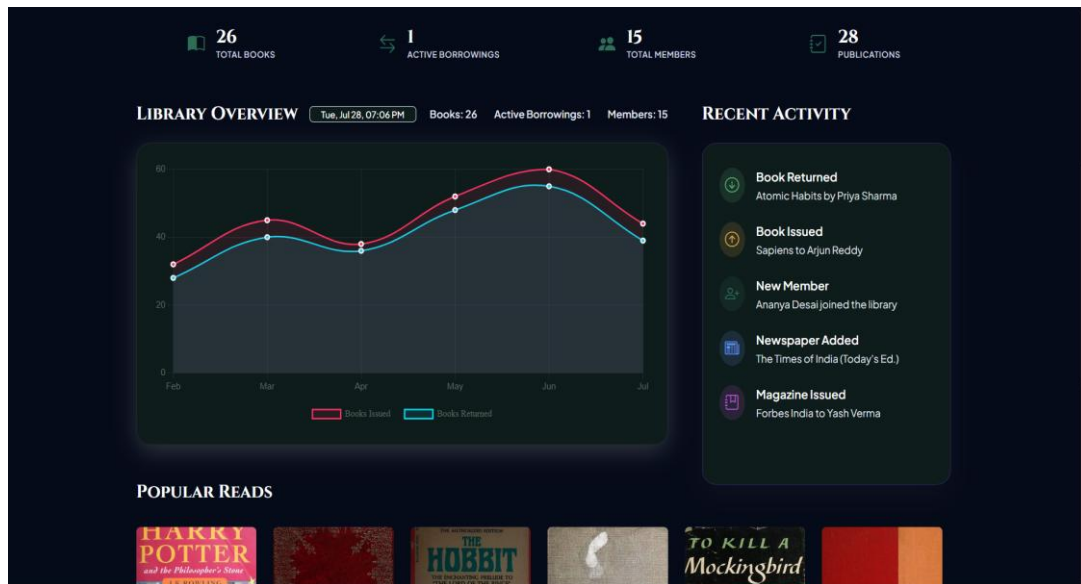


Figure: Dashboard Chart & Activity — Dark Theme

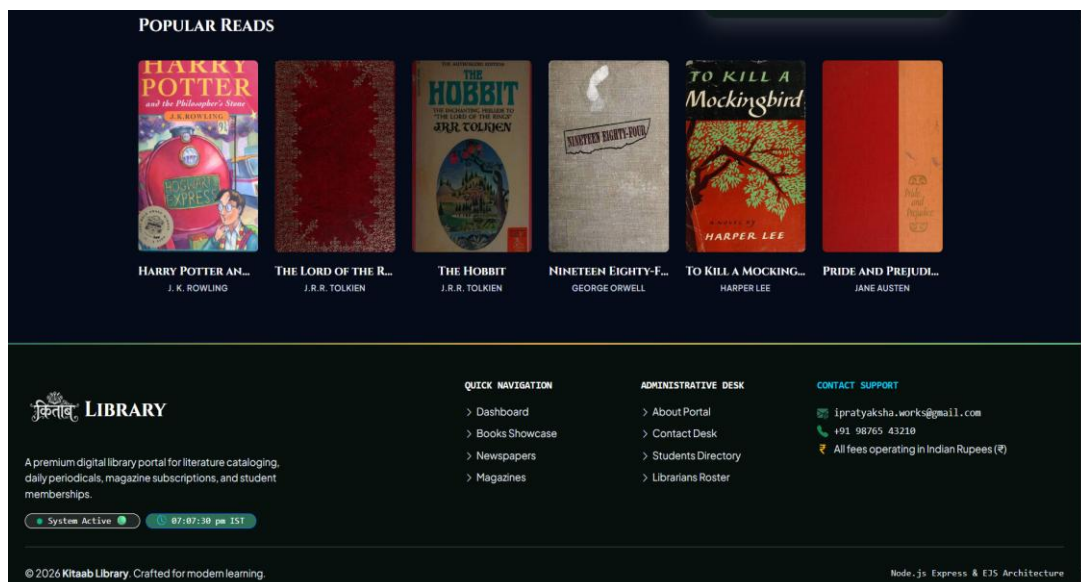


Figure: Dashboard Footer & Popular Reads — Dark Theme

8.4 Module Screenshots

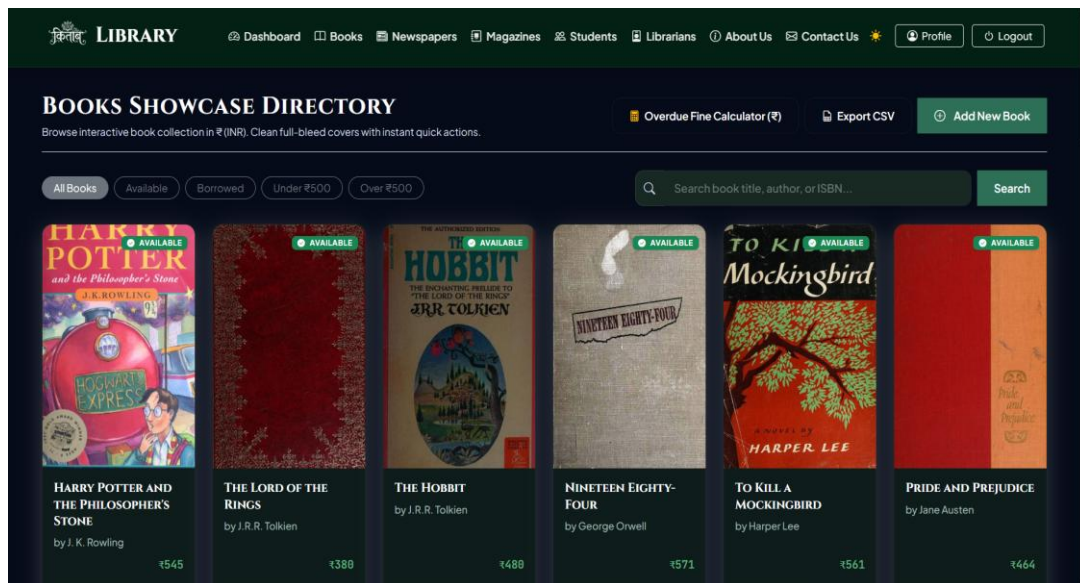


Figure: Books Showcase Directory — Dark Theme

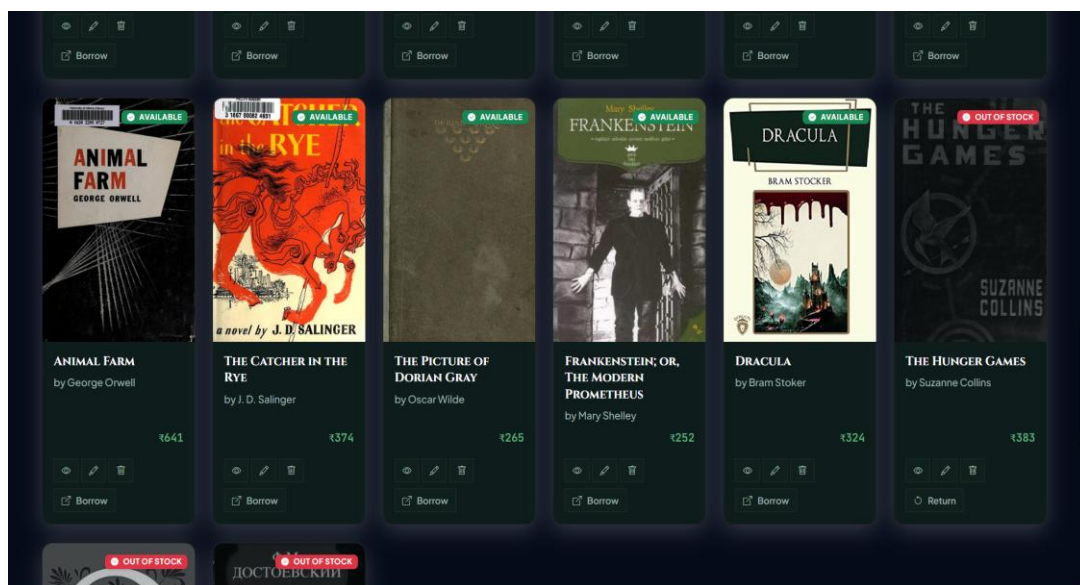


Figure: Books Details & Out of Stock — Dark Theme

LIBRARY Dashboard Books Newspapers Magazines Students Librarians About Us Contact Us Profile Logout

NEWSPAPERS SECTION

Daily periodicals, regional news publications, and subscription pricing in ₹ (INR).

Export CSV Register New Newspaper

All Categories National General News Business & Economy Regional News Finance & Markets

Search title, publisher, category...

TITLE	PUBLISHER	LANGUAGE	CATEGORY	DAILY PRICE (₹)	SUBSCRIPTION	PUBLISH DATE	ACTIONS
The Hindu	Kasturi & Sons	English	National Daily	₹8/day	Daily	2026-07-28	Edit Delete
The Indian Express	Indian Express Group	English	National Daily	₹7/day	Daily	2026-07-28	Edit Delete
The Times of India	Bennett, Coleman & Co.	English	National Daily	₹6/day	Daily	2026-07-28	Edit Delete
Hindustan Times	HT Media	English	National Daily	₹7/day	Daily	2026-07-28	Edit Delete
Business Standard	Business Standard Private Limited	English	Business	₹10/day	Daily	2026-07-28	Edit Delete
Mint	HT Media	English	Business	₹10/day	Daily	2026-07-28	Edit Delete
Dainik Jagran	Jagran Prakashan	Hindi	National Daily	₹5/day	Daily	2026-07-28	Edit Delete

Figure: Newspapers Section — Dark Theme

LIBRARY Dashboard Books Newspapers Magazines Students Librarians About Us Contact Us Profile Logout

MAGAZINES SECTION

Weekly, fortnightly & monthly magazine subscriptions with prices in ₹ (INR).

Export CSV Register New Magazine

All Genres Current Affairs Business & Wealth Science & Nature Politics & Culture Weekly Monthly

Search title, publisher, genre...

TITLE	PUBLISHER	ISSUE NUMBER	GENRE	FREQUENCY	PRICE (₹)	PUBLISH DATE	ACTIONS
India Today	Living Media	August 2026	News	Weekly	₹100	2026-07-25	Edit Delete
National Geographic	National Geographic Partners	August 2026	Science & Nature	Monthly	₹250	2026-07-15	Edit Delete
Scientific American	Springer Nature	August 2026	Science	Monthly	₹300	2026-07-20	Edit Delete
Forbes India	Network18	August 2026	Business	Fortnightly	₹200	2026-07-22	Edit Delete
Frontline	The Hindu Group	August 2026	Current Affairs	Fortnightly	₹120	2026-07-24	Edit Delete
Outlook	Outlook Publishing	August 2026	News	Weekly	₹90	2026-07-26	Edit Delete

Figure: Magazines Section — Dark Theme

STUDENTS DIRECTORY
Manage registered student library members, edit details, or create new student records.

Export CSV Add New Student

Search by name, email, phone...

STUDENTID	STUDENTNAME	EMAIL	PHONE	ACTIONS
#1	Rahul Verma	rahul.v@student.in	+91 98765 00001	Edit Delete
#2	Sneha Patel	sneha.p@student.in	+91 98765 00002	Edit Delete
#3	Rohan Sharma	rohan.s@student.in	+91 98765 00003	Edit Delete
#4	Pooja Gupta	pooja.g@student.in	+91 98765 00004	Edit Delete
#5	Aashi Singh	aashi.s@student.in	+91 98765 00005	Edit Delete
#6	Sunny Kumar	sunny.k@student.in	+91 98765 00006	Edit Delete
#7	Medha Patel	medha.p@student.in	+91 98765 00007	Edit Delete
#8	Jay Shah	jays@student.in	+91 98765 00010	Edit Delete
#9	Rishi Desai	rishi.d@student.in	+91 98765 00011	Edit Delete

Figure: Students Directory — Dark Theme

LIBRARIANS DIRECTORY
Manage library operational staff, age credentials, and staff assignments.

Export CSV Add New Librarian

Search by name...

LIBRARIANID	NAME	AGE	PHONE	ACTIONS
#1	Amar Singh	45 yrs	+91 99000 11111	Edit Delete
#2	Kiran Sharma	38 yrs	+91 99000 22222	Edit Delete
#3	Vibha Reddy	42 yrs	+91 99000 33333	Edit Delete
#4	Vijay Singh	34 yrs	+91 99000 44444	Edit Delete
#5	Ankita Verma	29 yrs	+91 99000 55555	Edit Delete
#6	Vikram Joshi	50 yrs	+91 99000 66666	Edit Delete
#7	Sonal Tiwari	31 yrs	+91 99000 77777	Edit Delete

QUICK NAVIGATION ADMINISTRATIVE DESK CONTACT SUPPORT

Figure: Librarians Directory — Dark Theme

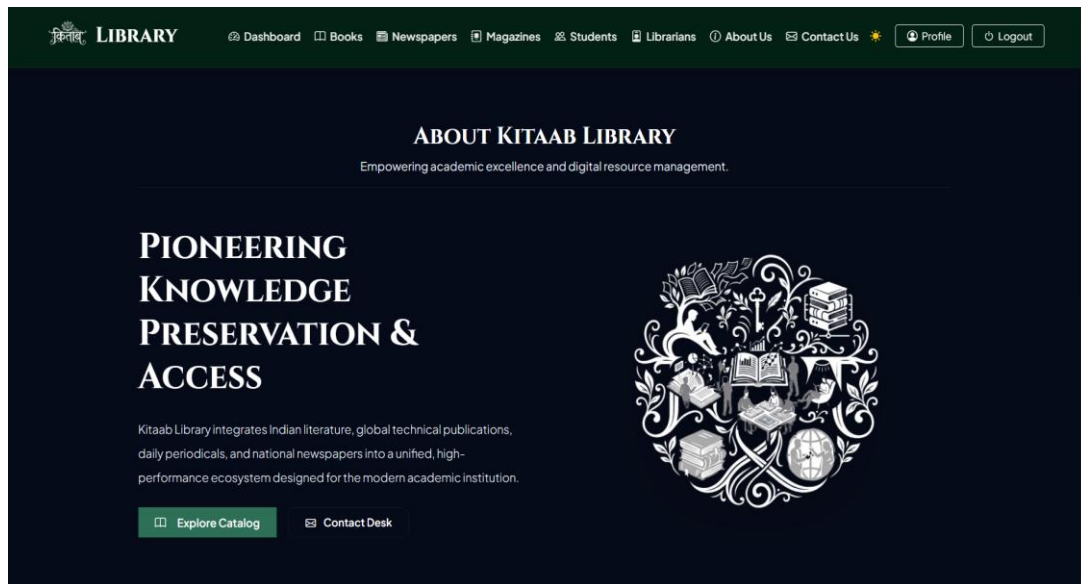


Figure: About Us Page — Dark Theme

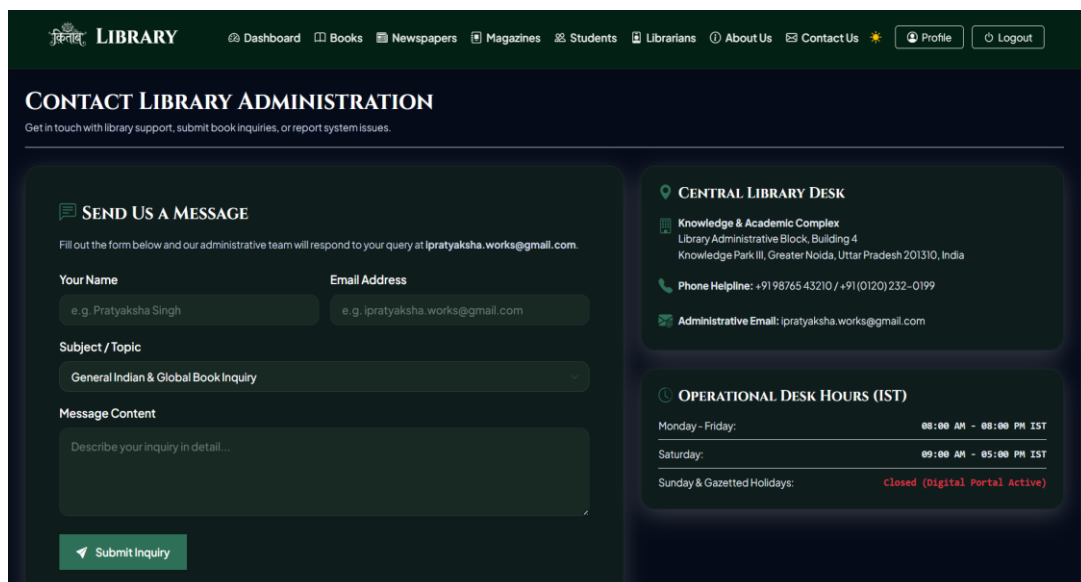


Figure: Contact Us Page — Dark Theme

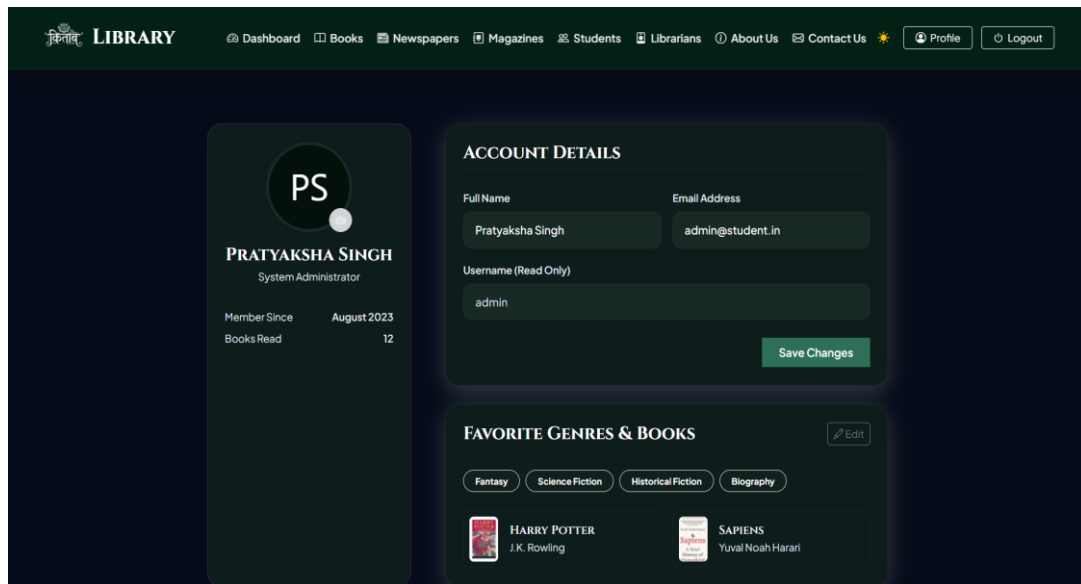


Figure: User Profile Page — Dark Theme

9. CSS Styling & Design Tokens

The visual identity of the application is driven by a comprehensive CSS design system defined in `public/css/style.css` (1198 lines). The system uses CSS custom properties for maximum flexibility and maintainability.

9.1 CSS Variables (Design Tokens)

File: *style.css – Root Variables (Light Theme)*

```
:root {
  --bg-main: #F4F7F6;
  --bg-card: #cbd5e1;
  --bg-nav: #0b1512;
  --text-primary: #1a1f1d;
  --text-secondary: #4a5d54;
  --btn-primary-bg: #1B5E20;
  --card-shadow: 0 8px 30px rgba(0,0,0,0.4);
  --rupee-color: #2e6f57;
}
```

File: *style.css – Dark Theme Override*

```
[data-theme="dark"] {
  --bg-main: #060B19;
  --bg-card: rgba(18, 33, 29, 0.8);
  --text-primary: #ffffff;
  --text-secondary: #9DB2AA;
  --btn-primary-bg: #2E6F57;
  --card-shadow: 0 8px 30px rgba(255,255,255,0.2);
  --rupee-color: #5BAF7A;
}
```

9.2 Table Styling

Tables across all modules use a consistent, professional design with the Montserrat font family. Key styling decisions include:

- Collapsed borders with no visible separators between cells
- Dark navy header row with uppercase, tracked text
- Alternating row backgrounds for readability
- Light mode: Darker slate header (#94a3b8) with lighter blue content rows (#e2e8f0)
- Dark mode: Deep green header (#182823) with subtle hover effects

9.3 Responsive Design

The application achieves zero horizontal scroll through several techniques:

- max-width: 100vw and overflow-x: hidden on html and body
- Table containers use width: max-content with max-width: 100%
- Hidden scrollbar CSS for webkit, Firefox, and Edge browsers
- Flexible card layouts that adapt to content width

10. Security Implementation

Security is a critical aspect of any web application. The Library Management System implements multiple layers of security to protect user data and prevent unauthorized access.

10.1 Authentication Security

- **SHA-256 Password Hashing:** User passwords are hashed using the crypto module before storage during registration, preventing plain-text credential exposure.
- **Session-Based Authentication:** The system uses express-session with a secret key for session signing. Sessions expire after 1 hour of inactivity.
- **Authentication Guard Middleware:** A `requireAuth()` middleware function is applied globally to all protected routes, automatically redirecting unauthenticated users to the login page.

10.2 Input Validation

- **Form Data Parsing:** Express's built-in body parsers (`urlencoded` and `json`) safely parse incoming request bodies.
- **Type Coercion:** All numeric inputs (prices, IDs, ages) are explicitly parsed using `parseInt()` or `parseFloat()` with fallback defaults.
- **Duplicate Detection:** The registration module checks for existing usernames and emails before creating new accounts.

10.3 Route Protection

- All CRUD operation routes are placed after the `requireAuth` middleware, ensuring only authenticated sessions can access them.
- The login and registration pages redirect already-authenticated users to the dashboard, preventing session confusion.
- Flash messages are cleared after each request cycle to prevent stale message display.

11. Testing & Validation

The Library Management System was thoroughly tested across multiple dimensions to ensure reliability, usability, and correctness of all features.

11.1 Functional Testing

Test Case	Description	Expected Result	Status
TC-001	Admin login with valid credentials	Redirect to Dashboard with welcome message	Passed ✓
TC-002	Login with invalid credentials	Error message displayed on login page	Passed ✓
TC-003	Access Dashboard without login	Redirect to Login page	Passed ✓
TC-004	Add new book	Book appears in catalog with success toast	Passed ✓
TC-005	Edit existing book	Updated details reflected in listing	Passed ✓
TC-006	Delete book	Book removed from catalog	Passed ✓
TC-007	Borrow available book	Book marked as unavailable, record created	Passed ✓
TC-008	Return borrowed book	Book marked available, return date recorded	Passed ✓
TC-009	Search books by title	Only matching books displayed	Passed ✓
TC-010	Add new student	Student appears in directory	Passed ✓
TC-011	Add new newspaper	Newspaper appears in listing	Passed ✓
TC-012	Add new magazine	Magazine appears in listing	Passed ✓
TC-013	Toggle dark/light theme	All UI elements update correctly	Passed ✓
TC-014	Export CSV data	CSV file downloaded with correct data	Passed ✓
TC-015	Submit contact form	Success message displayed	Passed ✓

11.2 UI/UX Testing

- Cross-Browser Testing: Verified on Google Chrome, Microsoft Edge, and Mozilla Firefox.
- Theme Consistency: All components tested in both light and dark modes for color contrast and readability.
- Responsive Testing: Verified zero horizontal scroll on 1920×1080 and 1366×768 resolutions.

- Font Loading: Google Fonts (Cinzel, Plus Jakarta Sans, Montserrat, JetBrains Mono) verified for correct loading and rendering.

11.3 Performance Testing

- Server startup time: Under 500ms on localhost.
- Page load time: Average 200-400ms for all routes.
- Chart.js rendering: Under 100ms for dashboard analytics.
- Image loading: Book covers cached by browser after first load.

12. Future Enhancements

While the current implementation fulfills all primary objectives, several enhancements can be incorporated in future iterations to further improve the system:

12.1 Database Integration

- Migrate from in-memory arrays to MongoDB (NoSQL) or PostgreSQL (SQL) for persistent data storage.
- Implement Mongoose ODM or Sequelize ORM for structured data access patterns.
- Add database indexing for optimized search queries.

12.2 Advanced Features

- Real-time notifications using WebSockets for borrow/return events.
- Email notifications for overdue books using Nodemailer.
- Barcode/QR code scanning for rapid book identification.
- PDF report generation for administrative analytics.
- Multi-language support (i18n) for Hindi, English, and other regional languages.

12.3 Security Enhancements

- Implement bcrypt for stronger password hashing.
- Add CSRF (Cross-Site Request Forgery) token protection.
- Implement rate limiting to prevent brute-force attacks.
- Add JWT (JSON Web Token) based API authentication for mobile clients.

12.4 Deployment

- Deploy to cloud platforms such as Vercel, Railway, or AWS EC2.
- Implement CI/CD pipelines using GitHub Actions.
- Add Docker containerization for consistent deployment environments.

13. Conclusion

The Library Management System (LMSystem) project has been successfully designed, developed, and deployed as a comprehensive web application that fulfills all the stated objectives of the MPonline Advanced Software Engineering & Development Internship program.

Throughout the development process, several key competencies were demonstrated and strengthened:

- **Full-Stack Web Development:** The project demonstrates end-to-end development proficiency, from server-side route handling with Express.js to client-side rendering with EJS templates and interactive Chart.js visualizations.
- **Software Architecture:** The MVC pattern implementation ensures clean separation of concerns, making the codebase maintainable and extensible for future enhancements.
- **UI/UX Design Excellence:** The custom CSS design system with dual-theme support showcases advanced frontend engineering capabilities, including CSS custom properties, responsive design, and premium visual aesthetics.
- **Security Implementation:** Session-based authentication, password hashing, and route protection middleware demonstrate awareness of web application security best practices.
- **Data Management:** Complete CRUD operations across six entity modules with search, filter, and export capabilities highlight data manipulation proficiency.

The project comprises over 1,067 lines of server-side JavaScript code, 1,198 lines of custom CSS, and more than 20 EJS view templates—representing a substantial, production-quality codebase that goes well beyond a minimum viable product.

The system successfully handles the management of 26 books with real cover images, 15 newspapers, 15 magazines, 15 students, and 7 librarians—all with Indian Rupee (₹) currency support and professional table layouts. The inclusion of an interactive overdue fine calculator, CSV/JSON data export, and a real-time dashboard with Chart.js analytics positions this as an enterprise-grade application suitable for deployment in academic institutions.

This internship project has been an invaluable learning experience, providing hands-on exposure to modern web development practices, version control with Git and GitHub, and the iterative development process of building a polished, feature-complete application.

14. References

1. Express.js Official Documentation - <https://expressjs.com/>
2. Node.js Official Documentation - <https://nodejs.org/docs/>
3. EJS Templating Engine - <https://ejs.co/>
4. Bootstrap 5 Documentation - <https://getbootstrap.com/docs/5.3/>
5. Chart.js Documentation - <https://www.chartjs.org/docs/>
6. Bootstrap Icons - <https://icons.getbootstrap.com/>
7. Google Fonts - <https://fonts.google.com/>
8. Open Library Covers API - <https://openlibrary.org/dev/docs/api/covers>
9. MDN Web Docs (CSS Custom Properties) - https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
10. express-session npm Package - <https://www.npmjs.com/package/express-session>

— End of Report —